

第12章 GPIO 输出—使用固件库点亮 LED

利用固件库建立好的工程模板，就可以方便地使用 STM32 标准库编写应用程序了，可以说从这一章我们才开始迈入 STM32 开发的大门。LED 灯的控制使用到 GPIO 外设的基本输出功能，本章中不再赘述 GPIO 外设的概念和功能框图的讲解，而是重点讲解如果用固件库编程。

12.1 硬件设计

在本教程中 STM32 芯片的 PF6、PF7 和 PF8 引脚分别与一个 RGB 灯的 R 灯（红灯）、G 灯（蓝灯）和 B 灯（绿灯）连接，具体见图 12-1。RGB 灯里面的三个小灯都可以单独控制，如果有两个或者三个灯同时亮的话就会混合成其它的颜色。

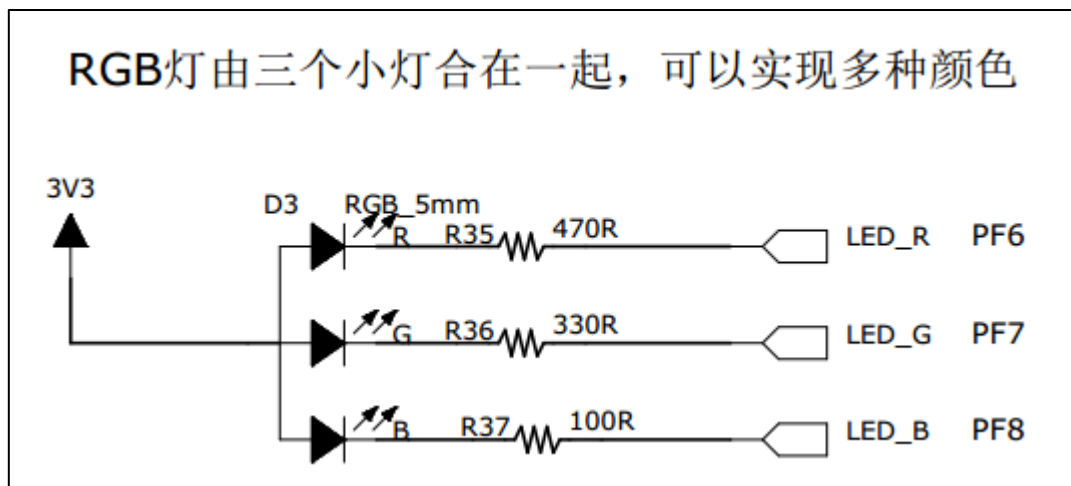


图 12-1 RGB 灯电路连接图

这些 LED 灯的阴极都是连接到 STM32 的 GPIO 引脚，只要我们控制 GPIO 引脚的电平输出状态，即可控制 LED 灯的亮灭。若你使用的实验板 LED 灯的连接方式或引脚不一样，只需根据我们的工程修改引脚即可，程序的控制原理相同。

12.2 软件设计

这里只讲解核心部分的代码，有些变量的设置，头文件的包含等可能不会涉及到，完整的代码请参考本章配套的工程。

为了使工程更加有条理，我们把 LED 灯控制相关的代码独立分开存储，方便以后移植。在“工程模板”之上新建“bsp_led.c”及“bsp_led.h”文件，其中的“bsp”即 Board Support Packet 的缩写(板级支持包)，这些文件也可根据您的喜好命名，这些文件不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

12.2.1 编程要点

1. 使能 GPIO 端口时钟；
2. 初始化 GPIO 目标引脚为推挽输出模式；
3. 编写简单测试程序，控制 GPIO 引脚输出高、低电平。

12.2.2 代码分析

1. LED 灯引脚宏定义

在编写应用程序的过程中，要考虑更改硬件环境的情况，例如 LED 灯的控制引脚与当前的不一样，我们希望程序只需要做最小的修改即可在新的环境正常运行。这个时候一般把硬件相关的部分使用宏来封装，若更改了硬件环境，只修改这些硬件相关的宏即可，这些定义一般存储在头文件，即本例子中的“bsp_led.h”文件中，具体见代码清单 12-1。

代码清单 12-1 LED 控制引脚相关的宏

```
1 //引脚定义
2 /*****
3 //R 红色灯
4 #define LED1_PIN                GPIO_Pin_6
5 #define LED1_GPIO_PORT          GPIOF
6 #define LED1_GPIO_CLK            RCC_AHB1Periph_GPIOF
7
8 //G 绿色灯
9 #define LED2_PIN                GPIO_Pin_7
10 #define LED2_GPIO_PORT          GPIOF
11 #define LED2_GPIO_CLK            RCC_AHB1Periph_GPIOF
12
13 //B 蓝色灯
14 #define LED3_PIN                GPIO_Pin_8
15 #define LED3_GPIO_PORT          GPIOF
16 #define LED3_GPIO_CLK            RCC_AHB1Periph_GPIOF
17 *****/
```

以上代码分别把控制四盏 LED 灯的 GPIO 端口、GPIO 引脚号以及 GPIO 端口时钟封装起来了。在实际控制的时候我们就直接用这些宏，以达到应用代码跟硬件无关的效果。

其中的 GPIO 时钟宏“RCC_AHB1Periph_GPIOH”和“RCC_AHB1Periph_GPIOD”是 STM32 标准库定义的 GPIO 端口时钟相关的宏，它的作用与“GPIO_Pin_x”这类宏类似，是用于指示寄存器位的，方便库函数使用。它们分别指示 GPIOH、GPIOD 的时钟，下面初始化 GPIO 时钟的时候可以看到它的用法。

2. 控制 LED 灯亮灭状态的宏定义

为了方便控制 LED 灯，我们把 LED 灯常用的亮、灭及状态反转的控制也直接定义成宏，具体见代码清单 12-2。

代码清单 12-2 控制 LED 亮灭的宏

```
1 /** 控制 LED 灯亮灭的宏，
2  * LED 低电平亮，设置 ON=0，OFF=1
3  * 若 LED 高电平亮，把宏设置成 ON=1，OFF=0 即可
4  */
5 #define ON 0
```

```
6 #define OFF 1
7
8 /* 带参宏, 可以像内联函数一样使用 */
9 #define LED1(a) if (a) \
10     GPIO_SetBits(LED1_GPIO_PORT, LED1_PIN); \
11     else \
12     GPIO_ResetBits(LED1_GPIO_PORT, LED1_PIN)
13
14 #define LED2(a) if (a) \
15     GPIO_SetBits(LED2_GPIO_PORT, LED2_PIN); \
16     else \
17     GPIO_ResetBits(LED2_GPIO_PORT, LED2_PIN)
18
19 #define LED3(a) if (a) \
20     GPIO_SetBits(LED3_GPIO_PORT, LED3_PIN); \
21     else \
22     GPIO_ResetBits(LED3_GPIO_PORT, LED3_PIN)
23
24
25 /* 直接操作寄存器的方法控制 IO */
26 #define digitalHi(p,i)    {p->BSRRL=i;}    //设置为高电平
27 #define digitalLo(p,i)    {p->BSRRH=i;}    //输出低电平
28 #define digitalToggle(p,i) {p->ODR ^=i;}    //输出反转状态
29
30 /* 定义控制 IO 的宏 */
31 #define LED1_TOGGLE    digitalToggle(LED1_GPIO_PORT, LED1_PIN)
32 #define LED1_OFF        digitalHi(LED1_GPIO_PORT, LED1_PIN)
33 #define LED1_ON         digitalLo(LED1_GPIO_PORT, LED1_PIN)
34
35 #define LED2_TOGGLE    digitalToggle(LED2_GPIO_PORT, LED2_PIN)
36 #define LED2_OFF        digitalHi(LED2_GPIO_PORT, LED2_PIN)
37 #define LED2_ON         digitalLo(LED2_GPIO_PORT, LED2_PIN)
38
39 #define LED3_TOGGLE    digitalToggle(LED3_GPIO_PORT, LED3_PIN)
40 #define LED3_OFF        digitalHi(LED3_GPIO_PORT, LED3_PIN)
41 #define LED3_ON         digitalLo(LED3_GPIO_PORT, LED3_PIN)
42
43 /* 基本混色, 后面高级用法使用 PWM 可混出全彩颜色, 且效果更好 */
44
45 //红
46 #define LED_RED \
47     LED1_ON; \
48     LED2_OFF; \
49     LED3_OFF
50
51 //绿
52 #define LED_GREEN \
53     LED1_OFF; \
54     LED2_ON; \
55     LED3_OFF
56
57 //蓝
58 #define LED_BLUE \
59     LED1_OFF; \
60     LED2_OFF; \
61     LED3_ON
62
63
64 //黄(红+绿)
65 #define LED_YELLOW \
66     LED1_ON; \
67     LED2_ON; \
68     LED3_OFF
69
70 //紫(红+蓝)
71 #define LED_PURPLE \
72     LED1_ON; \
```

```
72         LED2_OFF;\
73         LED3_ON
74
75 //青(绿+蓝)
76 #define LED_CYAN \
77         LED1_OFF;\
78         LED2_ON;\
79         LED3_ON
80
81 //白(红+绿+蓝)
82 #define LED_WHITE \
83         LED1_ON;\
84         LED2_ON;\
85         LED3_ON
86
87 //黑(全部关闭)
88 #define LEDRGBOFF \
89         LED1_OFF;\
90         LED2_OFF;\
91         LED3_OFF
```

这部分宏控制 LED 亮灭的操作是直接向 BSRR 寄存器写入控制指令来实现的，对 BSRR 写 1 输出高电平，对 BSRRH 写 1 输出低电平，对 ODR 寄存器某位进行异或操作可反转位的状态。

RGB 彩灯可以实现混色，如最后一段代码我们控制红灯和绿灯亮而蓝灯灭，可混出黄色效果。

代码中的“\”是 C 语言中的续行符语法，表示续行符的下一行与续行符所在的代码是同一行。代码中因为宏定义关键字“#define”只是对当前行有效，所以我们使用续行符来连接起来，以下的代码是等效的：

```
#define LED_YELLOW    LED1_ON; LED2_ON; LED3_OFF
```

应用续行符的时候要注意，在“\”后面不能有任何字符(包括注释、空格)，只能直接回车。

3. LED GPIO 初始化函数

利用上面的宏，编写 LED 灯的初始化函数，具体见代码清单 12-3。

代码清单 12-3 LED GPIO 初始化函数

```
1 void LED_GPIO_Config(void)
2 {
3     /*定义一个 GPIO_InitTypeDef 类型的结构体*/
4     GPIO_InitTypeDef GPIO_InitStructure;
5
6     /*开启 LED 相关的 GPIO 外设时钟*/
7     RCC_AHB1PeriphClockCmd ( LED1_GPIO_CLK |
8                               LED2_GPIO_CLK |
9                               LED3_GPIO_CLK, ENABLE);
10
11     /*选择要控制的 GPIO 引脚*/
12     GPIO_InitStructure.GPIO_Pin = LED1_PIN;
13
14     /*设置引脚模式为输出模式*/
15     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
16
17     /*设置引脚的输出类型为推挽输出*/
18     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
19 }
```

```
20  /*设置引脚为上拉模式*/
21  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
22
23  /*设置引脚速率为 2MHz */
24  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
25
26  /*调用库函数，使用上面配置的 GPIO_InitStructure 初始化 GPIO*/
27  GPIO_Init(LED1_GPIO_PORT, &GPIO_InitStructure);
28
29  /*选择要控制的 GPIO 引脚*/
30  GPIO_InitStructure.GPIO_Pin = LED2_PIN;
31  GPIO_Init(LED2_GPIO_PORT, &GPIO_InitStructure);
32
33  /*选择要控制的 GPIO 引脚*/
34  GPIO_InitStructure.GPIO_Pin = LED3_PIN;
35  GPIO_Init(LED3_GPIO_PORT, &GPIO_InitStructure);
36
37  /*关闭 RGB 灯*/
38  LED_RGBOFF;
39 }
```

整个函数与“自己写库—构建库函数雏形”章节中的类似，主要区别是硬件相关的部分使用宏来代替了，初始化 GPIO 端口时钟时也采用了 STM32 库函数，函数执行流程如下：

- (1) 使用 GPIO_InitTypeDef 定义 GPIO 初始化结构体变量，以便下面用于存储 GPIO 配置。
- (2) 调用库函数 RCC_AHB1PeriphClockCmd 来使能 LED 灯的 GPIO 端口时钟，在前面的章节中我们是直接向 RCC 寄存器赋值来使能时钟的，不如这样直观。该函数有两个输入参数，第一个参数用于指示要配置的时钟，如本例中的“RCC_AHB1Periph_GPIOH”和“RCC_AHB1Periph_GPIOD”，应用时我们使用“|”操作同时配置四个 LED 灯的时钟；函数的第二个参数用于设置状态，可输入“Disable”关闭或“Enable”使能时钟。
- (3) 向 GPIO 初始化结构体赋值，把引脚初始化成推挽输出模式，其中的 GPIO_Pin 使用宏“LEDx_PIN”来赋值，使函数的实现方便移植。
- (4) 使用以上初始化结构体的配置，调用 GPIO_Init 函数向寄存器写入参数，完成 GPIO 的初始化，这里的 GPIO 端口使用“LEDx_GPIO_PORT”宏来赋值，也是为了程序移植方便。
- (5) 使用同样的初始化结构体，只修改控制的引脚和端口，初始化其它 LED 灯使用的 GPIO 引脚。
- (6) 使用宏控制 RGB 灯默认关闭。

4. 主函数

编写完 LED 灯的控制函数后，就可以在 main 函数中测试了，具体见代码清单 12-4。

代码清单 12-4 控制 LED 灯，main 文件

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
3
4 void Delay(__IO u32 nCount);
5
6 /**
7  * @brief 主函数
8  * @param 无
9  * @retval 无
10  */
11 int main(void)
12 {
13     /* LED 端口初始化 */
14     LED_GPIO_Config();
15
16     /* 控制 LED 灯 */
17     while (1) {
18         LED1( ON );      // 亮
19         Delay(0xFFFFFFFF);
20         LED1( OFF );     // 灭
21
22         LED2( ON );      // 亮
23         Delay(0xFFFFFFFF);
24         LED2( OFF );     // 灭
25
26         LED3( ON );      // 亮
27         Delay(0xFFFFFFFF);
28         LED3( OFF );     // 灭
29
30         /*轮流显示 红绿蓝黄紫青白 颜色*/
31         LED_RED;
32         Delay(0xFFFFFFFF);
33
34         LED_GREEN;
35         Delay(0xFFFFFFFF);
36
37         LED_BLUE;
38         Delay(0xFFFFFFFF);
39
40         LED_YELLOW;
41         Delay(0xFFFFFFFF);
42
43         LED_PURPLE;
44         Delay(0xFFFFFFFF);
45
46         LED_CYAN;
47         Delay(0xFFFFFFFF);
48
49         LED_WHITE;
50         Delay(0xFFFFFFFF);
51
52         LEDRGBOFF;
53         Delay(0xFFFFFFFF);
54     }
55 }
56
57 void Delay(__IO uint32_t nCount)    //简单的延时函数
58 {
59     for (; nCount != 0; nCount--);
60 }
```

在 main 函数中，调用我们前面定义的 LED_GPIO_Config 初始化好 LED 的控制引脚，然后直接调用各种控制 LED 灯亮灭的宏来实现 LED 灯的控制。

以上，就是一个使用 STM32 标准软件库开发应用的流程。

12.3 下载验证

把编译好的程序下载到开发板并复位，可看到 RGB 彩灯轮流显示不同的颜色。

12.4 STM32 标准库补充知识

1. SystemInit 函数去哪了？

在前几章我们自己建工程的时候需要定义一个 SystemInit 空函数，但是在这个用 STM32 标准库的工程却没有这样做，SystemInit 函数去哪了呢？

这个函数在 STM32 标准库的“system_stm32f4xx.c”文件中定义了，而我们的工程已经包含该文件。标准库中的 SystemInit 函数把 STM32 芯片的系统时钟设置成了 168MHz，即此时 AHB1 时钟频率为 168MHz，APB2 为 84MHz，APB1 为 42MHz。当 STM32 芯片上电后，执行启动文件中的指令后，会调用该函数，设置系统时钟为以上状态。

2. 断言

细心对比过前几章我们自己定义的 GPIO_Init 函数与 STM32 标准库中同名函数的读者，会发现标准库中的函数内容多了一些乱七八糟的东西，见代码清单 12-5。

代码清单 12-5 GPIO_Init 函数的断言部分

```
1 void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)
2 {
3     uint32_t pinpos = 0x00, pos = 0x00 , currentpin = 0x00;
4
5     /* Check the parameters */
6     assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
7     assert_param(IS_GPIO_PIN(GPIO_InitStruct->GPIO_Pin));
8     assert_param(IS_GPIO_MODE(GPIO_InitStruct->GPIO_Mode));
9     assert_param(IS_GPIO_PUPD(GPIO_InitStruct->GPIO_PuPd));
10
11     /* ----- 以下内容省略，跟前面我们定义的函数内容相同----- */
```

基本上每个库函数的开头都会有这样类似的内容，这里的“assert_param”实际是一个宏，在库函数中它用于检查输入参数是否符合要求，若不符合要求则执行某个函数输出警告，“assert_param”的定义见代码清单 12-6。

代码清单 12-6 stm32f4xx_conf.h 文件中关于断言的定义

```
1
2 #ifndef USE_FULL_ASSERT
3 /**
4  * @brief assert_param 宏用于函数的输入参数检查
5  * @param expr:若 expr 值为假，则调用 assert_failed 函数
6  *             报告文件名及错误行号
7  *             若 expr 值为真，则不执行操作
8  */
9 #define assert_param(expr) \
10     ((expr) ? (void)0 : assert_failed((uint8_t *)__FILE__, __LINE__))
11 /* 错误输出函数 ----- */
12 void assert_failed(uint8_t* file, uint32_t line);
13 #else
```



```
14 #define assert_param(expr) ((void)0)
15 #endif
```

这段代码的意思是，假如我们不定义“USE_FULL_ASSERT”宏，那么“assert_param”就是一个空的宏(#else 与 #endif 之间的语句生效)，没有任何操作。从而所有库函数中的 assert_param 实际上都无意义，我们就当看不见好了。

假如我们定义了“USE_FULL_ASSERT”宏，那么“assert_param”就是一个有操作的语句(#if 与 #else 之间的语句生效)，该宏对参数 expr 使用 C 语言中的问号表达式进行判断，若 expr 值为真，则无操作(void 0)，若表达式的值为假，则调用“assert_failed”函数，且该函数的输入参数为“__FILE__”及“__LINE__”，这两个参数分别代表“assert_param”宏被调用时所在的“文件名”及“行号”。

但库文件只对“assert_failed”写了函数声明，没有写函数定义，实际用时需要用户来定义，我们一般会用 printf 函数来输出这些信息，见代码清单 12-7。

代码清单 12-7 assert_failed 输出错误信息

```
1 void assert_failed(uint8_t* file, uint32_t line)
2 {
3     printf("\r\n 输入参数错误，错误文件名=%s,行号=%s",file,line);
4 }
```

注意在我们的这个 LED 工程中，还不支持 printf 函数(在 USART 外设章节会讲解)，想测试 assert_failed 输出的读者，可以在这个函数中做点亮红色 LED 灯的操作，作为警告输出测试。

那么为什么函数输入参数不对的时候，assert_param 宏中的 expr 参数值会是假呢？这要回到 GPIO_Init 函数，看它对 assert_param 宏的调用，它被调用时分别以

“IS_GPIO_ALL_PERIPH(GPIOx)”、“IS_GPIO_PIN(GPIO_InitStruct->GPIO_Pin)”等作为输入参数，也就是说被调用时，expr 实际上是一条针对输入参数的判断表达式。例如“IS_GPIO_PIN”的宏定义：

```
1 #define IS_GPIO_PIN(PIN)    ((PIN) != (uint32_t)0x00)
```

若它的输入参数 PIN 值为 0，则表达式的值为假，PIN 非 0 时表达式的值为真。我们知道用于选择 GPIO 引脚号的宏“GPIO_Pin_x”的值至少有一个数据位为 1，这样的输入参数才有意义，若 GPIO_InitStruct->GPIO_Pin 的值为 0，输入参数就无效了。配合“IS_GPIO_PIN”这句表达式，“assert_param”就实现了检查输入参数的功能。对 assert_param 宏的其它调用方式类似，大家可以自己看库源码来研究一下。

3. Doxygen 注释规范

在 STM32 标准库以及我们自己编写的“bsp_led.c”文件中，可以看到一些比较特别的注释，类似代码清单 12-8。

代码清单 12-8 Doxygen 注释规范

```
1 /**
```



```
2 * @brief 初始化控制 LED 的 IO
3 * @param 无
4 * @retval 无
5 */
```

这是一种名为“Doxygen”的注释规范，如果在工程文件中按照这种规范去注释，可以使用 Doxygen 软件自动根据注释生成帮助文档。我们所说非常重要的库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》，就是由该软件根据库文件的注释生成的。关于 Doxygen 注释规范本教程不作讲解，感兴趣的读者可自行搜索网络上的资料学习。

4. 防止头文件重复包含

在 STM32 标准库的所有头文件以及我们自己编写的“bsp_led.h”头文件中，可看到类似代码清单 12-9 的宏定义。它的功能是防止头文件被重复包含，避免引起编译错误。

代码清单 12-9 防止头文件重复包含的宏

```
1 #ifndef __LED_H
2 #define __LED_H
3
4 /*此处省略头文件的具体内容*/
5
6 #endif /* end of __LED_H */
```

在头文件的开头，使用“#ifndef”关键字，判断标号“__LED_H”是否被定义，若没有被定义，则从“#ifndef”至“#endif”关键字之间的内容都有效，也就是说，这个头文件若被其它文件“#include”，它就会被包含到其该文件中了，且头文件中紧接着使用“#define”关键字定义上面判断的标号“__LED_H”。当这个头文件被同一个文件第二次“#include”包含的时候，由于有了第一次包含中的“#define __LED_H”定义，这时再判断“#ifndef __LED_H”，判断的结果就是假了，从“#ifndef”至“#endif”之间的内容都无效，从而防止了同一个头文件被包含多次，编译时就不会出现“redefine（重复定义）”的错误了。

一般来说，我们不会直接在 C 的源文件写两个“#include”来包含同一个头文件，但可能因为头文件内部的包含导致重复，这种代码主要是避免这样的问题。如“bsp_led.h”文件中使用了“#include“stm32f4xx.h””语句，按习惯，可能我们写主程序的时候会在 main 文件写“#include“bsp_led.h”及#include“stm32f4xx.h””，这个时候“stm32f4xx.h”文件就被包含两次了，如果没有这种机制，就会出错。

至于为什么要用两个下划线来定义“__LED_H”标号，其实这只是防止它与其它普通宏定义重复了，如我们用“GPIO_PIN_0”来代替这个判断标号，就会因为 stm32f4xx.h 已经定义了 GPIO_PIN_0，结果导致“bsp_led.h”文件无效了，“bsp_led.h”文件一次都没被包含。